

Uvicorn, FastAPI, Dotenv et Requests

Tutoriel sur la création d'API backend avec Python

Le Mans Université

Projet de troisième année de licence

Rédigé par :

REVERBEL-LONGHI Lucas

MORIN Nathan

THOURAULT Ilann



1 Introduction

L'objectif de ce tutoriel est de vous apprendre à construire une API en backend avec *Python*. Ces quatre bibliothèques sont une alternative intelligente à **NodeJS** avec **Express** par exemple. Notamment, *Python* permet une utilisation plus simple, tout en étant extrêmement extensible. Avec par exemple une simplicité pour ajouter de l'IA aux projets.

Pour cela, nous allons utiliser Uvicorn, FastAPI, Dotenv et Requests.

2 Dotenv

Dotenv permet de charger des variables d'environnement dans du code *Python*.

2.1 Initialisation

- Installation : avec la commande bash : `pip install python-dotenv`
- Importation :

```
from dotenv import load_dotenv
import os
```

La librairie `os` sera utilisée avec `Dotenv` pour lire les variables d'environnement.

Certaines APIs demandent des clefs d'authentification. Le problème est que ces clefs ne doivent pas apparaître sur votre dépôt git (pour des raisons de sécurité, elles doivent être transmises en privé). Il est conseillé d'utiliser un environnement public pour les usages qui peuvent être lu par tous, et un privé pour gérer des accès privés.

À partir d'ici nous considérerons deux fichiers d'environnement : `.public_env` et `.private_env`, en nous intéressant principalement au second fichier.

Une première version de `.private_env` peut être rendu public sur le dépôt, pour définir les noms et structures de ce fichier. Par exemple pour l'API de `scanR` :

```
SCANR_USERNAME=  
SCANR_PASSWORD=
```

De plus, il faut ajouter `.private_env` dans le `.gitignore` pour écarter tout danger.

2.2 Utilisation

Une fois l'importation faite et l'initialisation du fichier `.private_env`, nous pouvons maintenant montrer comment utiliser `Dotenv`.

Il y a un premier problème. On voudrait faire : `load_dotenv(".private_env")` mais *Python* charge les fichiers à partir de la racine de l'exécution. Des erreurs peuvent donc intervenir rapidement si le code est exécuté depuis la racine plutôt que depuis le répertoire *backend* par exemple.

La solution est d'utiliser la fonction `os.path.exists()` ! Voici un exemple d'implémentation :

```
if os.path.exists(".private_env"):  
    load_dotenv(".private_env")  
elif os.path.exists("../.private_env"):  
    load_dotenv("../.private_env")  
else:  
    print("Error: .private_env file not found")  
    exit()
```

Maintenant que `.private_env` est chargé, nous pouvons récupérer les champs requis pour la suite :

```
username = os.getenv("SCANR_USERNAME")  
password = os.getenv("SCANR_PASSWORD")
```

Il est possible que ces champs ne soient pas récupérables et dans ce cas, il faut le vérifier :

```
if username is None:  
    print("Error : Failed to fetch .private_env::SCANR_USERNAME")  
    exit()  
if password is None:  
    print("Error : Failed to fetch .private_env::SCANR_PASSWORD")  
    exit()
```

Ces deux variables sont donc maintenant utilisables, par exemple :

```
print(f"{username} : {password}")  
>> etu_le_mans_scanR : Aqkdjuu23_kjloiQ
```

3 Requests

Requests est une librairie permettant d'effectuer des requêtes, elle est extrêmement utilisée dans le domaine du WEB / API en python.

3.1 Initialisation

- Installation : avec la commande bash : `pip install requests`
- Importation :

```
import requests
import json
```

Avant d'effectuer des requêtes directement sous python, il est conseillé d'utiliser des outils comme **postman** pour bien se préparer.

3.2 Utilisation

Nous allons réutiliser `username` et `password`.

Pour des raisons de clarté, il est recommandé de définir l'URL et le header dans des variables, pour ne pas surcharger l'appel ensuite :

```
url = "https://cluster-production.elasticsearch.dataesr.ovh/scanr-organizations/_search"
headers = {
    "Content-Type": "application/json",
}
```

Il existe beaucoup de choses à faire avec cette librairie, nous allons vous montrer dans ce tutoriel comment effectuer une méthode POST, avec la fonction `requests.post()`.

```
response = requests.post(
    url,
    auth=(username, password),
    headers=headers,
    data=json.dumps({
        "query": {
            "query_string": {
                "fields": ["address.city"],
                "query": "\"Le Mans\""
            }
        },
        "from": 0,
        "size": 100
    })
)
```

Cet exemple permet de récupérer sur l'API de scanR, les 100 premiers lieux situés au Mans. Le contenu du résultat de la requête est contenu dans la variable `response`.

```
print("Status code:", response.status_code)
print("Response body:", response.json()["hits"]["hits"][0])
```

Cette première ligne, affiche le code de retour, par exemple **404** ou **200** si tout c'est bien passé. La deuxième ligne affiche le corps de la réponse. `response.json()` renvoie un dictionnaire, il est donc facile de l'explorer avec la fonction `.keys()`.

Tout ce dont vous pourriez avoir besoin est inscrit dans la documentation [1]

4 FastAPI



FastAPI est une librairie *Python* permettant la création d'une API.

4.1 Initialisation

— Installation : `pip install fastapi[standard]`

— Importation :

```
from fastapi import FastAPI, Response
from fastapi.middleware.cors import CORSMiddleware
```

De plus, il faut initialiser FastAPI, de cette façon : `app = FastAPI()`

Le nom donné à cette variable est important pour la suite.

Si votre API sera utilisée par un autre langage que *Python*, il faut alors définir un middleware CORS (*Cross-Origin Resource Sharing*).

Nous travaillons ici avec angular sur le port 8080, nous devons donc indiquer au middleware d'accepter le *localhost* sur ce port.

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:8080"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Les paramètres indiqués ne sont pas prévus pour la production, il faudra les vérifier et les adapter avant l'utilisation dans des cas réels.

4.2 Utilisation

Voici comme exemple, une requête qui envoie tous les points GPS des organisations ayant collaborées dans un projet inclus dans les dates passées en paramètre sur l'API HAL.

```
@app.get("/hal/getCoordinatesFromDates")
def getCoordinatesFromDates(anneeMin: int, anneeMax: int, moisMin: int, moisMax: int):
    return hal.getCoordinatesFromDates(anneeMin, anneeMax, moisMin, moisMax)
```

Le décorateur `@app` correspond au nom donné au moment de l'initialisation de FastAPI.

Le paramètre donné à `@app.get()` correspond au chemin dans l'URL, par exemple ici :

`http://localhost:8080/hal/getCoordinatesFromDates/`.

Les différents paramètres de la requête sont directement récupérés par la fonction inscrite juste en dessous, tant que les noms dans l'URL correspondent à ceux des arguments. Ils peuvent

néanmoins, être indiqué explicitement.

La recherche de correspondance entre l'URL et les méthodes se fait dans l'ordre de déclaration des fonctions. Si une fonction n'est pas trouvée, une erreur 404 sera renvoyée.

Toute la documentation est disponible ici : [2]

5 Uvicorn



Uvicorn est une *Asynchronous Server Gateway Interface*. Nous l'utilisons pour lancer notre serveur.

5.1 Initialisation

— Installation : `pip install uvicorn`

5.2 Utilisation

— Exécution : `uvicorn main:app --reload --port 4000`

`--reload` sert à relancer automatique l'exécution lors d'un changement dans le code.

`--port` précise le port sur lequel votre API sera disponible. Il est conseillé d'inscrire le port dans le `.public_env` pour qu'il soit commun au front-end et au back-end.

`app` correspond au nom avec lequel vous avez instancié **FastAPI** dans le code.

Toute la documentation est inscrite ici : [3]

6 Bibliographie

Références

[1] *Documentation officielle de la librairie requests* <https://requests.readthedocs.io/>

[2] *Documentation officielle de la librairie FastAPI* <https://fastapi.tiangolo.com/>.

[3] *Documentation officielle de la librairie Uvicorn* <https://uvicorn.dev/>.